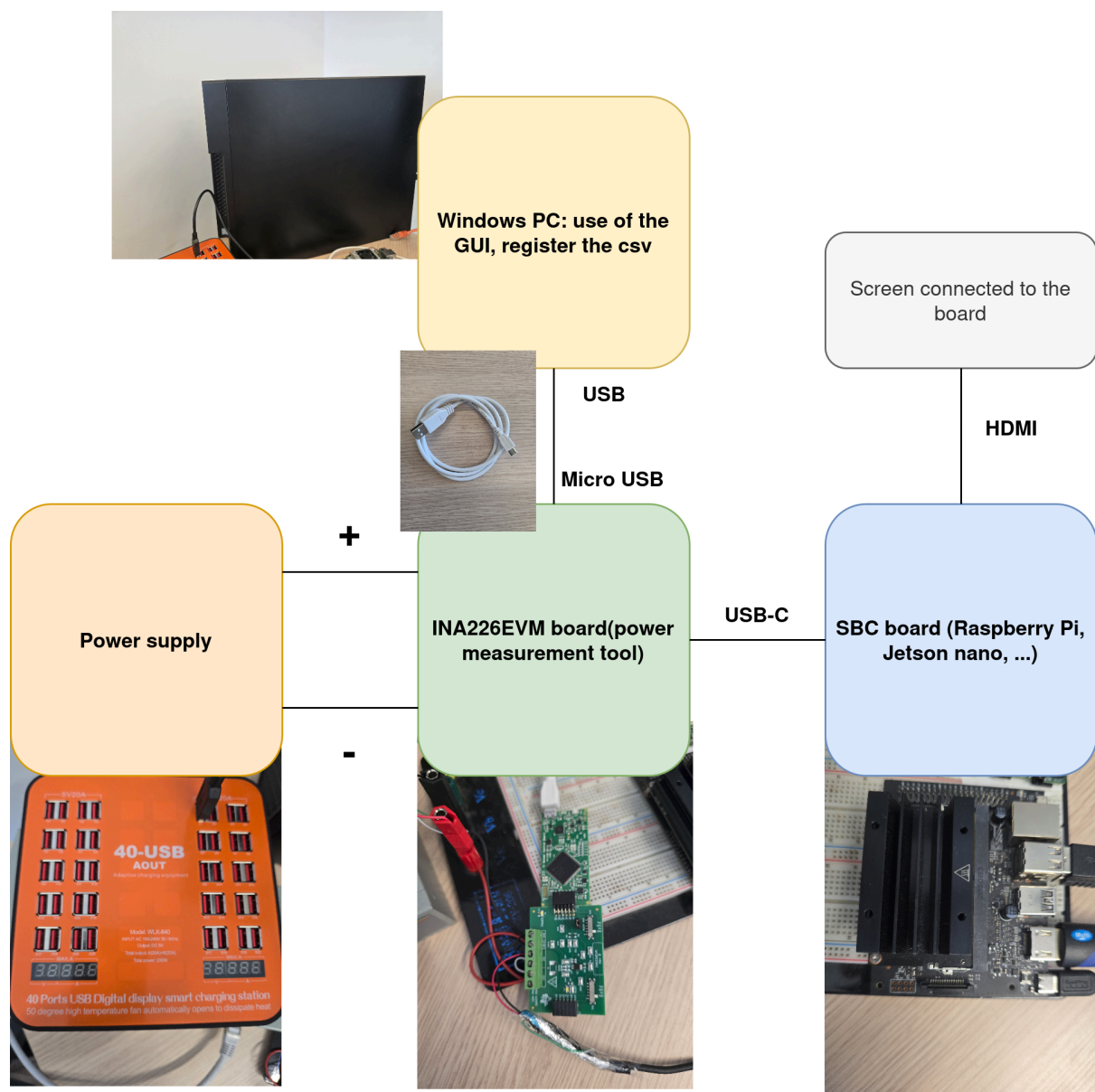


BANERA power measurement protocol

Context	1
Set up	2
Measurement to perform	6
Report generation	7
Future work	8
TIPS	9

Context

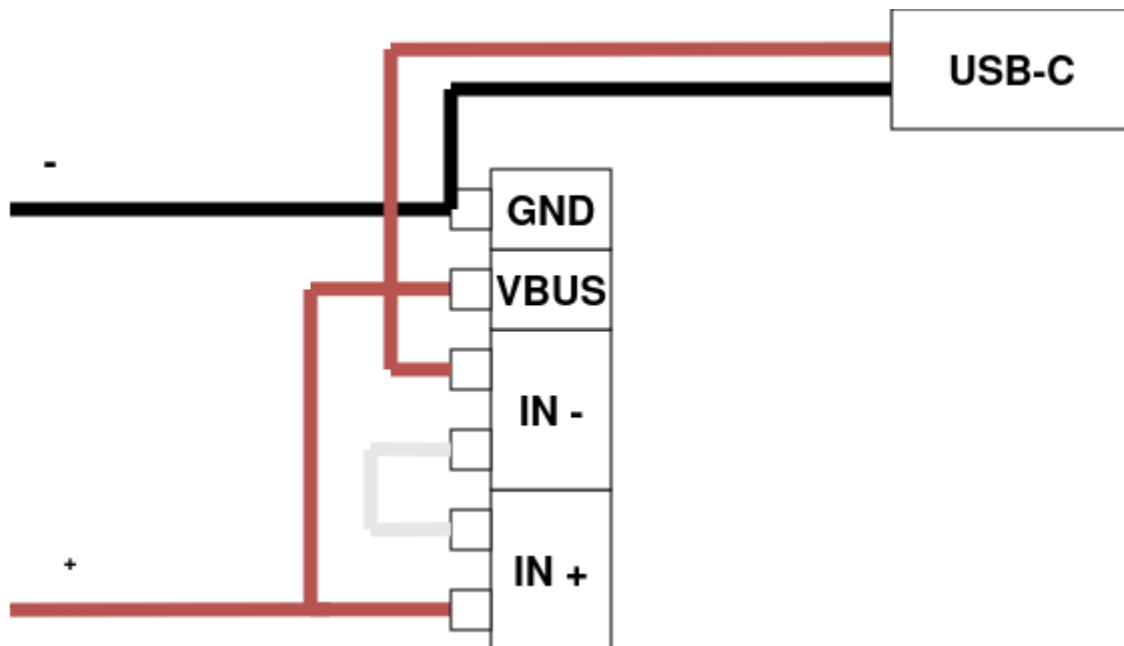


The above scheme describe the connections between the different components used to perform the power measurement:

First the power supply is connected to the measurement device (INA226EVM board).

Then the measurement device is connected to the board on which the measure are performed. (Raspberry Pi 5, Jetson nano, Jetson nano Orin, or Coral Dev Board). The INA226EVM board is also connected via USB to a Windows pc.

Here is a detailed view of the connection on the INA226EVM board:



Set up

Open the Windows PC (Donald)

Open Firefox

Click in the link in preferences:

dev.ti.com/gallery/view/CurrentSensing/INA226EVM_GUI/ver/1.0.0/

INA226EVM_GUI

Once the page is loaded close the pop-up windows, the INA226EVM will connect automatically:

README.md

This GUI requires the INA226EVM and an SCB Controller.

To start, please plug the EVM board into your computer's USB port and close this README.md file.

The GUI should automatically connect with your EVM.

- You can click the  button in the status bar at the bottom of the GUI to connect to the EVM.
- Once connected, clicking on the  button in the status bar will disconnect from the EVM.

To see this readme again once it has been closed, please select Help / View README.md

☐ Don't show again!

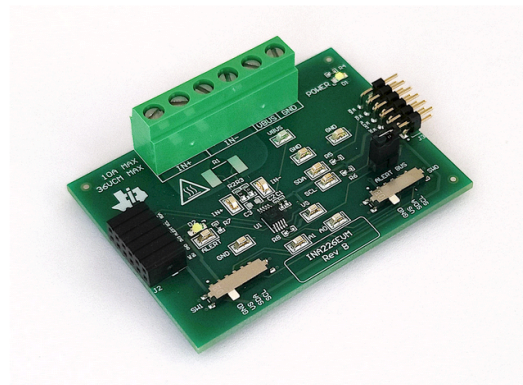
CLOSE



DEVICE CONNECTING...

 **INA226**

The INA226 is a current shunt and power monitor with an I2C™ or SMBUS-compatible interface. The device monitors both a shunt voltage drop and bus supply voltage. Programmable calibration value, conversion times, and averaging, combined with an internal multiplier, enable direct readouts of current in amperes and power in watts. [Read more](#)



 Functional Diagram

 User Guide

 Datasheet

 Schematic

 E2E Forum

One connected click on the gear icon on the left

In the field RSHUNT, put the value of the **12mOhm**. This is the value of the shunt resistor which is the white thread on the figure.

EVM 1

Hardware Switch Settings	
SW1:	GND ▾
SW0:	GND ▾

Shunt/Current Configuration	
RSHUNT:	12  mΩ
Max Expected Current:	3  A
CURRENT_LSB:	0.000091552734375 A
CALIBRATION:	4660
True CURRENT_LSB:	0.00009155937052932762 A
True Max Current:	3.0002174535050075 A
Required Sensing Range:	36 mV

On the field max expected current put the max expected current of the board you want to measure. To get the information check the datasheet of the board. For the Coral Dev Board and the Jetson Nano it is **3A**.

Go to the graphic window.

Put the sampling rate from 2.2 ms to 100ms

Put the number of sample from 1000 to 10000 samples

▼

COLLECT DATA

☐ #Samples Only

Delay between samples:

2.200 ms, ☒ Auto Delay

SAVE ALL PLOTS

CLEAR ALL PLOTS

#Samples: 1000

(Changing #Samples clears all plots)

]

COLLECT DATA

☐ #Samples Only

Delay between samples:

100.000 ms, ☐ Auto Delay

SAVE ALL PLOTS

CLEAR ALL PLOTS

#Samples: 10000

(Changing #Samples clears all plots)

On the board put the following file :

- run_manager.py
- runner.py
- base_runner.py

This file is important in order to launch the inferences sequence on the measured board.

On every thing is ready, you can start the measurement onto the Windows PC, by clicking onto start collect, and also launch the inference on the board.

In the terminal board launch the following command:

For PyTorch:

- **Linear layer(Linear_*inputSize*_outputSize*):**

```
python3 run_manager.py --backend cuda --model Linear_64_64 --nb_run 10 --sleep_time 100
```

- **Conv layer(conv_*inputChannels*_imageSize*_kernelSize*_padding*):**

```
python3 run_manager.py --backend cuda --model conv_1_32_3_0 --nb_run 10 --sleep_time 100
```

For TensorFlow:

Before launching the inference, you have to create the models. For this run the following command:

- For the linear model (go check model_builder_manager.py)

```
sizes = [64,128,256,512,1024,2048,4096,8192]
specs = list(product(sizes, sizes))

manager = ModelBuilderManager(TFTPULinearModelBuilder, specs)
```

- For the convolutional layer: an equivalent of linear_model_builder.py file as to be created

```
python3 run_manager.py --backend tpu --model Compiled/Linear_64_64_edgetpu.tflite
--nb_run 10 --sleep_time 100
```

On the measurement are finished, click on stop collect.

Then save all plot.

A file will be automatically downloaded onto the Windows office. Rename it from main.csv to:

- **Linear_*inputSize*_*outputSize***
- **Conv_*inputChannels*_*imageSize*_*kernelSize*_*padding***

Measurement to perform

Linear measurement :

	64	128	256	512	1024	2048	4096	8192
64								
128								
256								
512								
1024								
2048								

4096								
8192								

Convolutional measurement :

The line represent the image size ((32 by 32), (64 by 64), ...)

The column represent the input channel size (1 input, channel, 2 input channels,...)

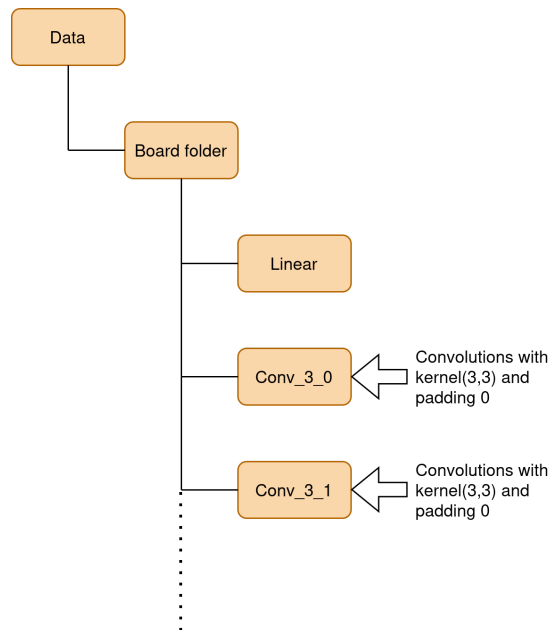
The measurement have to be performed for following padding and kernel size:

- k=(3,3), p=0
- k=(3,3), p=1
- k=(5,5), p=0
- k=(5,5), p=1

	32	64	128	256	512	1024
1						
2						
4						
8						
16						
32						
64						
128						
256						
512						

Report generation

Once all the measurement are all performed, you have to copy all the CSV files on a USB key and transfer them into the Data folder. In the data folder, each board as its own folder and inside it a folder for each type of measurement.



To build the final report, you have to indicate in the file `data_report.py` the path of the transferred csv. You have also to indicate the plot folder to register the plots associated with each csv files:

```
data_folder =  
"/home/frederic/Documents/BANERA/Code/Data/Jetson_nano_power_record/Conv_3_0"  
plot_folder =  
"/home/frederic/Documents/BANERA/Code/Plot/Jetson_nano_plots/Conv_3_0"
```

Once the script is done the report is created at the root of the project, and you can copy it into the result folder.

Command to generate the Excel report(use **banera_tf** env):

Python data_report.py

Future work

- Create the file `conv_model_builder.py` in order to be able to create model for convolution layer inference on TPU.

- Right now here are the state of the measurement (a cross in the cell means the measurement are done)
- Write python code to generate convolutionnal layer for TPU (Coral Dev Board)
- Automatization des test via protocol I2C

	Linear	Conv K = (3,3) P=0	Conv K = (3,3) P=1	Conv K = (5,5) P=0	Conv K = (5,5) P=1
Coral Dev Board	x				
Jetson Nano	x	x			
Jetson Nano Orin					
Raspberry Pi5					

All the code to generate linear/conv layer is already written. (jetson nano, jetson nano orin, Raspberry pi)

The code to generate the linear model for tensorflow is ready (Coral dev board).

The code to generate the conv layer has to be written .

TIPS

For some really demanding layer inference, it is better to run only 2 inferences max, and one the first one is finished just register the data and stop the program, otherwise it is too long to wait for the result.

The power supply needs to be changed, because it starts not working very well on the last week of my work.

This is the power supply for the Rasperrypi 5:

